

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 3, Section 3.1
- **Estimated reading time:** 15 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 1

WORKING WITH STOCK RETURNS

Derivative Pricing - Module 5



Outline

- ▶ Using APIs to access financial data
- ▶ Dealing with stock prices and returns: Tips and best practices

Working with stock price data

In this first lesson of Module 5, we will start dealing with real data on stock returns.

⇒ Go to the Jupyter Notebook associated with this lesson to explore all these topics.

Some of these issues are covered in the Jupyter Notebook:

- ▶ Access application programming interface (API) to obtain data on stock prices
- ▶ Dealing with stock returns: simple vs. continuous compounding
- ▶ Main properties of simple vs. continuously compounded returns
- ▶ Annualized returns

Summary of Lesson 1

In Lesson 1, we have looked at:

- ▶ How to use an API to access financial data
- ▶ How to treat financial data and best practices associated with different calculations

⇒ **TO DO NEXT:** Now, please go to the Jupyter Notebook for this lesson if you haven't done so yet.

⇒ In the next lesson, we will take a closer look at the main stylized facts of historical stock returns.

DERIVATIVE PRICING

MODULE 5 | LESSON 1

WORKING WITH STOCK RETURNS

Reading Time	55 minutes
Prior Knowledge	Financial time series, Stock returns
Keywords	Financial data, Historical stock returns, ETFs, Compounding

In this lesson, we are going to introduce some basic features for downloading, treating, and working with financial data in practice. These are practices we will be using for the remainder of the course (as well as in the Stochastic Modeling course), so make sure you understand the different issues!

There are two main topics that we will cover in this notebook:

- How to work with financial data (stock prices) in Python.*
- How to calculate, treat, and work with stock return data in practice.*

Let's start with the first topic on reading financial data. All rights reserved WQU WorldQuant University

1. Introduction to Financial Data and Returns

In this section, we introduce how to work with financial data using real historical prices from equities and ETFs (Exchange-Traded Funds). Instead of relying on unstable APIs, we use data stored in CSV files, which is a common and reliable method for reproducible analysis.

We will work with the following tickers:

- **SPY**: ETF tracking the S&P 500 Index (large-cap U.S. equities).
- **IWM**: ETF tracking the Russell 2000 Index (small-cap U.S. equities).
- **KO**: Stock of The Coca-Cola Company (consumer staples sector).
- **TSLA**: Stock of Tesla, Inc. (automotive and technology sector).

We will load the data, visualize prices, and compute returns as a first step in derivative pricing models.

```
# Load necessary libraries
import pandas as pd
import matplotlib.pyplot as plt
```

```
# Load data from CSV files
spy = pd.read_csv('SPY_wqu_data.csv', parse_dates=['timestamp'])
iwm = pd.read_csv('IWM_wqu_data.csv', parse_dates=['timestamp'])
ko = pd.read_csv('KO_wqu_data.csv', parse_dates=['timestamp'])
tsla = pd.read_csv('TSLA_wqu_data.csv', parse_dates=['timestamp'])
```

```
for df in [spy, iwm, ko, tsla]:
    df.set_index('timestamp', inplace=True)
    df.sort_index(inplace=True)
```

Understanding the Financial Data Structure

Before diving into returns and pricing models, it's important to understand the structure of the data we are working with.

Let's take a closer look at the SPY dataset, which contains historical prices of the SPY ETF (tracking the S&P 500 Index). The dataset includes daily price and volume information.

```
# Display the first few rows of SPY data
spy.head()
```

timestamp	volume	vw	open	close	high	low	transactions
2003-09-10 04:00:00	45417600.0	102.2064	102.530	101.96	102.80	101.55	25951
2003-09-11 04:00:00	38038300.0	102.3182	102.100	102.26	102.76	101.84	22580
2003-09-12 04:00:00	41159600.0	101.9165	101.910	102.45	102.64	101.35	26091
2003-09-15 04:00:00	20710600.0	102.2676	102.520	102.09	102.63	101.95	15670
2003-09-16 04:00:00	37103300.0	103.0466	102.232	103.58	103.64	102.17	19074

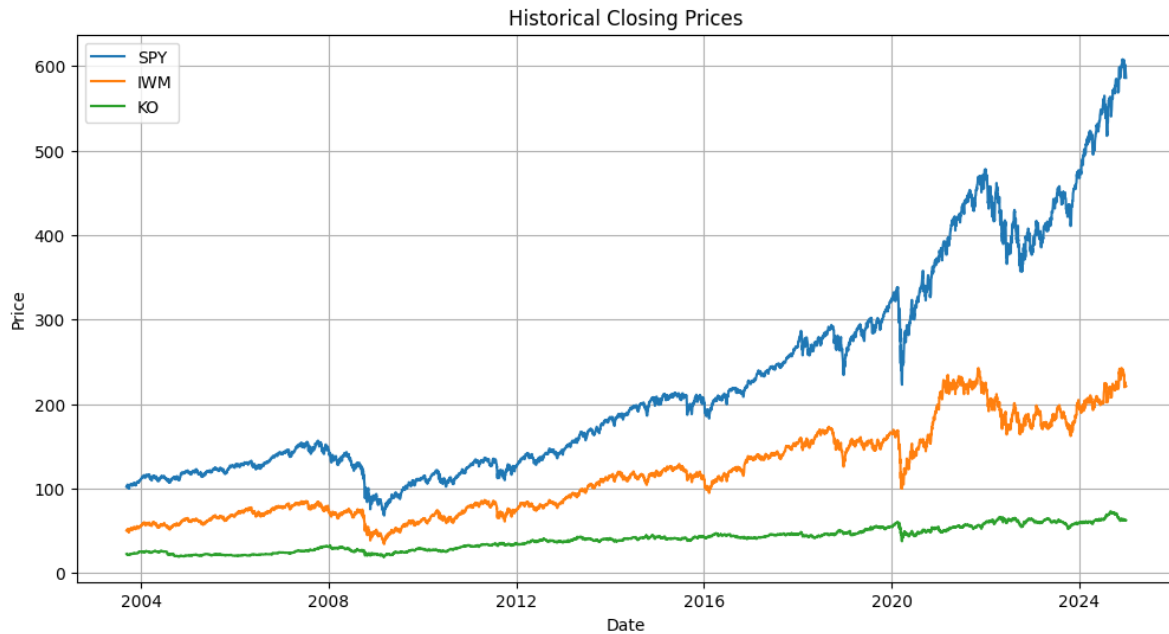
Here is a brief explanation of each column in the dataset:

- **open:** Price at which the ETF first traded when the market opened on that day.
- **close** (*adjusted close*): The final trading price of the day, adjusted for events like dividends and stock splits. This adjustment ensures that historical price comparisons remain meaningful over time.
- **high:** The highest price reached during the trading session.
- **low:** The lowest price reached during the trading session.
- **vw** (volume-weighted average price): An average price weighted by the trading volume throughout the day. It reflects the true average transaction price.
- **volume:** Total number of shares traded on that day.
- **transactions:** Number of individual trades that occurred during the day.

Important: In this dataset, the **close** column already reflects **adjusted close** prices. This is crucial for return calculations, as it accounts for dividends and splits, giving us a more accurate picture of the asset's performance over time.

Now, let's start by plotting the historical closing prices (adjusted for splits and dividends) for SPY, KO, and IWM.

```
plt.figure(figsize=(12, 6))
plt.plot(spy['close'], label='SPY')
plt.plot(iwm['close'], label='IWM')
plt.plot(ko['close'], label='KO')
plt.title('Historical Closing Prices')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.show()
```



Since TSLA stock has less historical data available, we conclude this simple exercise by plotting its adjusted closing prices.

```
plt.figure(figsize=(12, 6))
plt.plot(tsla['close'], label='TSLA')
plt.title('Historical Closing Prices')
plt.xlabel('Date')
plt.ylabel('Price')
plt.legend()
plt.grid(True)
plt.show()
```



2. Working with Stock Returns

First, let's look at the big picture regarding the types of financial data out there:

1. **Time Series data:** Data collected over a period of time on one or more variables.
2. **Cross-sectional data:** Cross-sectional data are data on one or more variables collected at a single point in time.
3. **Panel data:** Panel data have the dimensions of both time series and cross-sections, e.g., the daily prices of a number of blue chip stocks over two years.

Now, let's see how to look at stock returns:

2.1 Returns in Financial Modeling

There are two methods used to calculate returns from a series of prices, and these involve the calculation of simple returns or continuously compounded returns.

- Simple returns:

$$R_t = \frac{p_t - p_{t-1}}{p_{t-1}} \times 100\%$$

- Continuously compounded returns:

$$r_t = \ln\left(\frac{p_t}{p_{t-1}}\right) \times 100\%$$

(Review) [Continuous and Discrete compounding](#).

Importantly, these two methods yield different outcomes, which we need to be aware of.

```
import numpy as np

price_0 = 9 # Imagine this is the price at t=0
price_1 = 10 # Imagine this is the price at t=1

# So, returns for the period t=0 to t=1 will be:

Simple_ret = (price_1 - price_0) / price_0 * 100
Cont_ret = np.log(price_1 / price_0) * 100

print("The simple return at t=1 will be ", "{:.4f}".format(Simple_ret), "%")
print(
    "The continuously compounded return at t=1 will be ", "{:.4f}".format(Cont_ret), "%"
)
```

The simple return at t=1 will be 11.1111 %

The continuously compounded return at t=1 will be 10.5361 %

2.2 Returns: Simple Compounding

We assume that the asset is purchased at time $t - K$ for price P_{t-K} and then sold K periods later at price P_t .

$$R_{Kt} = \frac{P_t - P_{t-K}}{P_{t-K}} = \frac{P_t}{P_{t-K}} - 1 \rightarrow P_t = P_{t-K}(1 + R_{Kt}) \quad (1)$$

In the case of $K = 1$:

$$R_t = \frac{P_t - P_{t-1}}{P_{t-1}} = \frac{P_t}{P_{t-1}} - 1 \rightarrow P_t = P_{t-1}(1 + R_t) \quad (2)$$

The max and min values would be:

$$\begin{aligned} \min(R_t) &= -1 = -100\% \\ \max(R_t) &= +\infty \end{aligned} \quad (3)$$

2.3 Returns: Using Logs (continuously compounded)

We assume that the asset is purchased at time $t - K$ for price P_{t-K} and then sold K periods later at price P_t .

$$r_t^K = \ln \left(\frac{P_t}{P_{t-K}} \right) = \ln(P_t) - \ln(P_{t-K}) \longrightarrow P_t = P_{t-K} \exp r_t^K \quad (4)$$

In the case of $K = 1$:

$$r_t = \ln \left(\frac{P_t}{P_{t-1}} \right) = \ln(P_t) - \ln(P_{t-1}) \longrightarrow P_t = P_{t-1} \exp r_t \quad (5)$$

The max and min values would be:

$$\begin{aligned} \min(R_t) &= -\infty \\ \max(R_t) &= +\infty \end{aligned} \quad (6)$$

2.4 Returns: Properties

Let's assume that the asset is purchased at time $t - K$ for price P_{t-K} and then sold K periods later at price P_t . Then, if we calculate simple returns for each period, $R_t, R_{t+1}, \dots, R_K$, the aggregate return over all K periods is:

$$\begin{aligned} R_{Kt} &= \frac{P_t - P_{t-K}}{P_{t-K}} = \frac{P_t}{P_{t-K}} - 1 = \left[\frac{P_t}{P_{t-1}} \times \frac{P_{t-1}}{P_{t-2}} \dots \frac{P_{t-K-1}}{P_{t-K}} \right] - 1 = \\ &= [(1 + R_t)(1 + R_{t-1}) \dots (1 + R_{t-K+1})] = \prod_{j=0}^{K-1} (1 + R_{t-j}) \end{aligned} \quad (7)$$

For Log-returns:

$$\begin{aligned} r_t^K &= \ln \left(\frac{P_t}{P_{t-K}} \right) = \ln(P_t) - \ln(P_{t-K}) = \\ &= [\ln(P_t) - \ln(P_{t-1})] + \dots + [\ln(P_{t-(K-1)}) - \ln(P_{t-K})] \end{aligned} \quad (8)$$

2.5 Annualized Returns

If the asset was held for k years, then the annualized (average) return is defined as:

$$\text{Annualized } R_t[k] = \left[\prod_{j=0}^{k-1} (1 + R_{t-j}) \right]^{1/k} - 1 \quad (9)$$

This is a geometric mean of the k one-period simple gross returns involved and can be computed by:

$$\text{Annualized } R_t[k] = \exp \left[\frac{1}{k} \sum_{j=0}^{k-1} \ln(1 + R_{t-j}) \right] - 1 \quad (10)$$

Example: Buffet's Returns - How to use the annualized return

Period	Berkshire Hathaway	S&P500	Annual Outperf.
1960s	28.3%	5.0%	23.3%
1970s	22.2%	5.9%	16.3%
1980s	39.1%	17.3%	21.7%
1990s	20.5%	18.0%	2.5%
2000s	5.9%	-1.0%	6.8%
2010s	17.9%	15.3%	2.6%
1965 - 2014	21.7%	9.8%	11.8%

If you invested 100.000 euros in 1965, what is the amount of money in 2014? (Using the average annual return)

```
A_ret = 0.217
n_year = 2014 - 1965
Initial_invest = 100000

Total_ret = (1 + A_ret) ** n_year
Final_invest = Initial_invest * Total_ret
print("Your investment will be worth " + str(Final_invest / 10**6) + " millions!")
```

Your investment will be worth 1510.90908737206 millions!

3. Conclusion

Well done! This has been a quick review of concepts that, hopefully, you were already familiar with.

Now, jump to the next lesson to see a practical investigation of the different properties and stylized facts associated with stock returns.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

LESSON 2

STYLIZED FACTS OF STOCK RETURNS

Derivative Pricing - Module 5



Outline

- ▶ Stylized facts of stock returns: normality? heavy tails?
- ▶ Testing for normality: Jarque Bera, T-test,...

Stylized facts of stock returns

Most financial models assume **some normally distributed component for stock returns**.

Black-Scholes model and SDE, for example, assume that stock returns follow a normal distribution of type $\mathcal{N}(\mu, \sigma^2)$

- ▶ How realistic is this assumption?

In the Jupyter Notebook for this lesson, we will cover some of these questions in order to check the potential flaws of models such as Black-Scholes in certain situations.

This will be the first step towards modifying the BS canonical model to incorporate these issues in future models.

⇒ Go to the **Jupyter Notebook** for this lesson to explore all these issues.

Summary of Lesson 2

In this lesson, we have looked at:

- ▶ Treating financial data and best practices associated with different calculations.
- ▶ Checking the properties and stylized facts of stock returns.

⇒ **TO DO NEXT:** Now, please go to the associated Jupyter Notebook for this lesson if you haven't done so yet.

⇒ In the next lesson, we will learn how to deal with correlations between assets and how to simulate correlated returns under a GBM process.

DERIVATIVE PRICING

MODULE 5 | LESSON 2

STYLIZED FACTS OF STOCK RETURNS

Reading Time	70 min
Prior Knowledge	Normal distribution, Black-Scholes, Volatility
Keywords	Kurtosis, Skewness, T-distribution, Volatility, Correlation

In this lesson, we are going to investigate the different characteristics of historical stock returns. Specifically, we will focus on three main dimensions:

- **Normality / Log-normality** of stock returns and prices(?)
- **Volatility** of stock returns and the different ways to estimate it(?)
- **Correlation** between stock returns and its time-varying nature(?)

We will start by revisiting some concepts from statistics, which will help us provide answers to these questions later on.

1. Statistical Distributions: Mean and Standard Deviation (Std)

The l – th moment of a continuous random variable X is defined as

$$m_l = E[X^l] = \int_{-\infty}^{\infty} x^l f(x) dx, \quad (1)$$

where E stands for expectation and $f(x)$ is the probability density function of X . The first moment is called the **mean or expectation** of X . It measures the central location of the distribution. **We denote the mean of X by μ_x .**

The l – th central moment of X is defined as,

$$m_l = E[(X - \mu_x)^l] = \int_{-\infty}^{\infty} (x - \mu_x)^l f(x) dx, \quad (2)$$

The second central moment, denoted by σ_x^2 measures the variability of X and is called the variance of X . The positive square root, σ_x , of variance is the standard deviation of X . The first two moments of a random variable uniquely determine a normal distribution.

For other distributions, higher order moments are also of interest.

1.1. Higher Moments: Skewness and Kurtosis

The third central moment measures the symmetry of X with respect to its mean, whereas the fourth central moment measures the tail behavior of X . In statistics, **skewness** and **kurtosis**, which are the normalized third and fourth central moments of X , are often used to summarize the extent of asymmetry and tail thickness. Specifically, the standardized skewness and kurtosis of X are defined as,

$$S(X) = \frac{E[(X - \mu_n)^3]}{(E[(X - \mu_n)^2])^{3/2}} \quad (3)$$

$$K(X) = \frac{E[(X - \mu_n)^4]}{(E[(X - \mu_n)^2])^2} \quad (4)$$

The quantity $K(x) - 3$ is called the **excess kurtosis** because $K(x) = 3$ for a normal distribution. Thus, the excess kurtosis of a normal random variable is zero.

You can refresh these concepts and read more on this [here](#) and also [here](#).

Now, let's check all these on a real distribution of past stock returns.

```
# Load necessary libraries
import pandas as pd
import matplotlib.pyplot as plt
```

```
tsla = pd.read_csv('TSLA_wqu_data.csv', parse_dates=['timestamp'])
tsla.set_index('timestamp', inplace=True)
tsla.sort_index(inplace=True)
tsla.tail()
```

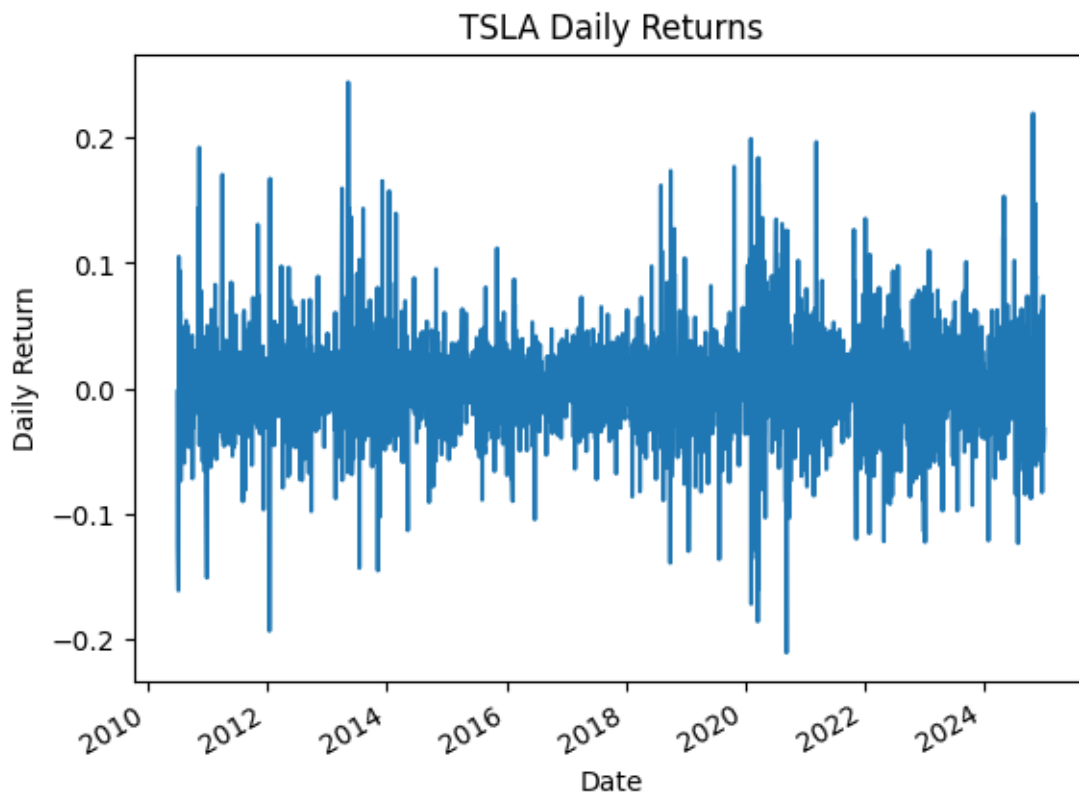
timestamp	volume	vw	open	close	high	low	transactions
2024-12-24 05:00:00	59551750.0	451.6202	435.90	462.28	462.7800	435.14	936767
2024-12-26 05:00:00	76651210.0	457.0321	465.16	454.13	465.3299	451.02	1251616
2024-12-27 05:00:00	82650021.0	434.9602	449.52	431.66	450.0000	426.50	1352470
2024-12-30 05:00:00	64924467.0	420.6715	419.40	417.41	427.0000	415.75	1178086
2024-12-31 05:00:00	76821539.0	412.9250	423.79	403.84	427.9300	402.54	1214552

These are Tesla stock prices from Jun. 6st, 2010, until the end of 2024. Let's now look at returns:

```
tsla_daily_returns = (tsla["close"] / tsla["close"].shift(1)) - 1

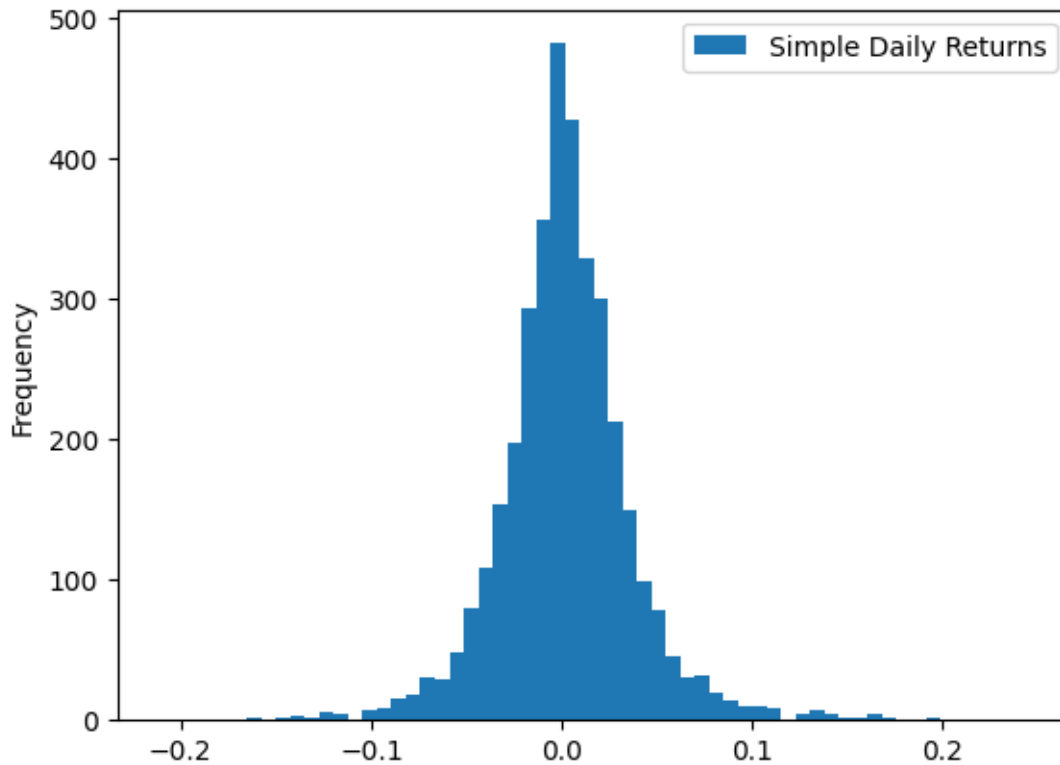
tsla_daily_returns_frame = tsla_daily_returns.to_frame()
tsla_daily_returns_frame.columns = ["Simple Daily Returns"]

tsla_daily_returns_frame["Simple Daily Returns"].plot()
plt.xlabel("Date")
plt.ylabel("Daily Return")
plt.title("TSLA Daily Returns")
plt.show()
```



... and the histogram of these to get a sense of the distribution:

```
tsla_daily_returns_frame.plot.hist(bins=60)  
plt.show()
```



What are the characteristics of this distribution?

```
avg_returns_daily = tsla_daily_returns.mean()
std_returns_daily = tsla_daily_returns.std()
print("The mean of the distribution is = " + str(avg_returns_daily))
print("The std. deviation (volatility) is = " + str(std_returns_daily))
```

The mean of the distribution is = 0.0021662259883025795

The std. deviation (volatility) is = 0.036144686759468865

What about skewness and kurtosis?

```
from scipy.stats import kurtosis, skew

array_tsla = tsla_daily_returns.to_numpy()
kurt = kurtosis(array_tsla[1:])
sk = skew(array_tsla[1:])
print("Kurtosis = " + str(kurt))
print("Skew = " + str(sk))
```

```
Kurtosis = 4.898686248325264
Skew = 0.3705196195429463
```

2. Do These Statistics Coincide with Those from a Normal Distribution?

Probably not. This is what we are checking next, with some formal tests.

Before jumping there, you may take a look at the following piece of news from [The Guardian](#), from the big banking crisis of 2007-2008.

2.1. Jarque Bera Test (Normality Test)

The Jarque-Bera test checks whether the sample data has the skewness and kurtosis that match a normal distribution.

The null hypothesis in the Jarque-Bera test is that the data follows a normal distribution. We are going to use the `scipy` library to simplify the tasks of performing this test.

[Here is the documentation](#) in case you want to check it.

```
import numpy as np
from scipy import stats

# Let's generate 10,000 random numbers following a Normal distribution
x = np.random.randn(10000)

# And check whether the Jarque-Bera test:
jb = stats.jarque_bera(x)
jb
```

```
SignificanceResult(statistic=np.float64(0.9500525229256406), pvalue=np.float64(0.621868725068))
```

The first output of the test is the Jarque-Bera statistic. The second is the associated p-value. Here, as the p-value is well above standard significance levels (0.01, 0.05 or 0.1), we cannot reject the null that this is a normal distribution.

Let's check the same for our daily stock returns from TSLA:

```
# We first need to get rid of the NaN values
tsla_ret = tsla_daily_returns_frame.dropna()

# Then, we perform the test
jb_tsla = stats.jarque_bera(tsla_ret)
jb_tsla
```

```
SignificanceResult(statistic=np.float64(3734.1007325003743), pvalue=np.float64(0.0))
```

As you can see here, the p-value is highly significant (0.0). We can thus reject the null hypothesis of returns following a normal distribution.

3. T-distribution and T-test

We already know kurtosis is a measure of whether the data are **heavy-tailed** or **light-tailed** relative to a normal distribution. That is, datasets with high kurtosis tend to have heavy tails, or outliers. Data sets with low kurtosis tend to have light tails, or a lack of outliers.

One possibility we have observed in practice is that stock returns' distribution have **heavy tails** (i.e., there are certain periods—crisis, shocks—in which we observe large returns: positive or negative. These outliers could make returns resemble a **Student's t-distribution**. The T-distribution is symmetric and bell-shaped, like the normal distribution, but has **heavier tails**, meaning that it is more prone to producing values that fall far from its mean. This makes it useful for understanding the statistical behavior of certain types of ratios of random quantities, in which variation in the denominator is amplified and may produce outlying values when the denominator of the ratio falls close to zero.

[More info on the T-distribution can be found here.](#)

3.1. Inferential Statistics: T-test (comparing means)

One useful test to determine whether our distribution follows a T-distribution is the **T-test**.

3.1.1. One-Sample T-Test

This test checks whether the mean of a normally-distributed sample is different from a pre-specified value:

- *Null Hypothesis*: Sample mean is equal to some value μ_0 .
- *Alternative Hypothesis*: Sample mean is different from μ_0 .

Let's perform this test on our sample in order to check whether the mean of MSFT daily returns is actually zero:

```
from scipy import stats

stats.ttest_1samp(tsla_ret, popmean=0)
```

```
TtestResult(statistic=array([3.62130591]), pvalue=array([0.0002971]), df=array([3650]))
```

In this case, we see that the p-value is 0.0002971 and, thus, we reject the null hypothesis. We conclude that the mean of returns is statistically different from zero.

3.1.2. Two-Sample T-Test

This tests whether the means of two populations/samples are significantly different from one another. The intuition is the same as before, but in this case, the value μ_0 is the mean from another sample.

Here, you have a more advanced reading in case you are interested: [‘How to code the student t-test from scratch in python’](#)

Let's do a quick example with 2 samples of daily returns:

```
# We read some data from KO (Coca-Cola) to compare mean returns to TSLA:
ko = pd.read_csv('KO_wqu_data.csv', parse_dates=['timestamp'])
ko.set_index('timestamp', inplace=True)
ko.sort_index(inplace=True)
ko_daily_returns = (ko["close"] / ko["close"].shift(1)) - 1
ko_daily_returns_frame = ko_daily_returns.to_frame()
ko_ret = ko_daily_returns_frame.dropna()
```

```
ko_ret
```

	close
timestamp	
2003-09-11 04:00:00	-0.010456
2003-09-12 04:00:00	-0.011016
2003-09-15 04:00:00	-0.011139
2003-09-16 04:00:00	-0.013333

timestamp	close
2003-09-17 04:00:00	0.002097
...	...
2024-12-24 05:00:00	0.007374
2024-12-26 05:00:00	-0.004297
2024-12-27 05:00:00	-0.001918
2024-12-30 05:00:00	-0.006725
2024-12-31 05:00:00	0.003708

```
from scipy import stats

stats.ttest_ind(tsla_ret, ko_ret)
```

```
TtestResult(statistic=array([3.61924502]), pvalue=array([0.00029708]), df=array([9012.]))
```

Please note that the previous calculations are just for illustrative purposes. There are a bunch of statistical properties (some of them that we will deal with next) that determine the suitability of this test. For example, we have done this test assuming the returns of both KO and TSLA are independent (not related), which may be too big of an assumption. There is a form of the 2-sample t-test, though, that allows us to consider related variables. For a more complete description of all the features, [you can check the scipy.stats.ttest library documentation](#).

4. Volatility of Stock Returns

Next, let's briefly look at the volatility of stock returns. There are 3 essential dimensions in which we can measure volatility:

- **Historical Volatility:** What has been the historical volatility of stock returns in the past X period. This is the one we will consider for some tests today.
- **Realized Volatility:** The actual volatility that stock returns will exhibit in the future X period. Obviously, this one we can just proxy for.
- **Implied Volatility:** We can extract the volatility expectations of investors from exchange-traded option prices. More on this in the next module.

As you already know, we measure volatility via the standard deviation of stock returns. If we measure it with *daily* stock returns, then we need to annualize the volatility as follows:

$$\sigma_T = \sqrt{T}\sigma_t$$

For instance, if the daily standard deviation of stock returns is 0.01%, the annualized volatility, σ , will be:

$$\sigma = \sqrt{252} \times 0.0001 = 0.1587\%$$

with $T = 252$ being the standard number of trading days in a year. Thus, we can calculate the volatility of stocks as the standard deviation of daily returns and annualize it later.

Would this volatility be the same for any period we consider?

```
# Ensure the index is datetime (if not already)
tsla_daily_returns.index = pd.to_datetime(tsla_daily_returns.index)

# Filter returns for each period
tsla_returns_2012_2017 = tsla_daily_returns['2012':'2017'].dropna()
tsla_returns_2018_2024 = tsla_daily_returns['2018':'2024'].dropna()

# Calculate annualized volatility
tsla_vol_2012_2017 = tsla_returns_2012_2017.std() * np.sqrt(252)
tsla_vol_2018_2024 = tsla_returns_2018_2024.std() * np.sqrt(252)

# Display the results
print(f"Tesla Annualized Volatility (2012-2017): {tsla_vol_2012_2017:.2%}")
print(f"Tesla Annualized Volatility (2018-2024): {tsla_vol_2018_2024:.2%}")
```

TSLA Annualized Volatility (2012-2017): 48.42%

TSLA Annualized Volatility (2018-2024): 63.70%

It seems annualized volatility exhibits a time-varying behavior, probably something we would expect ex-ante. The problem related with this is: **what period should we consider for the volatility parameters in GBM estimations?**

In short: None. We will consider **implied volatility** from option prices as a forward-looking measure of volatility. But don't worry, we will dig much deeper into this topic in the following modules!

5. Covariance between Stock Returns

Finally, let's deal with covariance between past stock returns. So far, we have used the GBM process to generate price paths for a single stock. In the next lesson, we will see how to generate

price paths for multiple stocks, which requires us to assume some correlation between them, and use some mathematical techniques like *Cholesky decomposition* to obtain a proper solution.

Before jumping there, however, let's briefly investigate the issue in practice. Let's rely on the **covariance matrix** as a metric capturing the statistical relationship between the returns of two stocks, specifically, we compute the daily covariance between TSLA and KO over two periods:

- 2012–2017: A period of post-crisis recovery and expansion
- 2018–2024: Including market disruptions from the pandemic and changing interest rate regimes

```
# Ensure datetime index for both series
tsla_daily_returns.index = pd.to_datetime(tsla_daily_returns.index)
ko_daily_returns.index = pd.to_datetime(ko_daily_returns.index)

# Filter returns for each period
tsla_2012_2017 = tsla_daily_returns['2012':'2017'].dropna()
ko_2012_2017 = ko_daily_returns['2012':'2017'].dropna()

tsla_2018_2024 = tsla_daily_returns['2018':'2024'].dropna()
ko_2018_2024 = ko_daily_returns['2018':'2024'].dropna()

# Align the indexes to ensure proper pairwise comparison
aligned_12_17 = pd.concat([tsla_2012_2017, ko_2012_2017], axis=1).dropna()
aligned_18_24 = pd.concat([tsla_2018_2024, ko_2018_2024], axis=1).dropna()

# Compute covariance
cov_12_17 = aligned_12_17.cov().iloc[0, 1]
cov_18_24 = aligned_18_24.cov().iloc[0, 1]

# Display results
print(f"Covariance between TSLA and KO (2012-2017): {cov_12_17:.6f}")
print(f"Covariance between TSLA and KO (2018-2024): {cov_18_24:.6f}")
```

Covariance between TSLA and KO (2012-2017): 0.000038

Covariance between TSLA and KO (2018-2024): 0.000075

As was the case with volatility, it seems that correlation between stock returns also exhibits a time-varying nature. This issue will be hard to deal with (which correlation should we consider?), but we will start looking at it by learning how to simulate correlated stock prices in the next lesson.

6. Conclusion

In this lesson, we have empirically documented some statistical features of the distribution of stock prices and returns. Ideally, our models to simulate asset prices should incorporate them all, although this would be virtually impossible. In the next lesson, we will start considering some of these issues by learning how to generate correlated simulated stock prices. All rights reserved WQU WorldQuant University QQQQ

References

- Brownlee, Jason. “How to Code the Student’s t-Test from Scratch in Python”. *Machine Learning Mastery*, 30 July 2018, <https://machinelearningmastery.com/how-to-code-the-students-t-test-from-scratch-in-python/>
- “Kurtosis.” *Wikipedia: The Free Encyclopedia*. Wikimedia Foundation, Inc, 4 July 2022, <https://en.wikipedia.org/wiki/Kurtosis>
- “Skewness.” *Wikipedia: The Free Encyclopedia*. Wikimedia Foundation, Inc, 4 July 2022, <https://en.wikipedia.org/wiki/Skewness>
- Stewart, Ian. “The Mathematical Equation That Caused the Banks to Crash”. *The Guardian*, 12 February 2012, <https://www.theguardian.com/science/2012/feb/12/black-scholes-equation-credit-crunch#:~:text=It%20was%20the%20holy,still%20had%20time%20to%20run.&text=tion%20itself%20wasn't%20the%20real%20problem>
- “Student’s t-distribution.” *Wikipedia: The Free Encyclopedia*. Wikimedia Foundation, Inc, 4 July 2022, https://en.wikipedia.org/wiki/Student%27s_t-distribution

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.

Required Readings

Required Readings for this program are open access, which means you should be able to access them at no cost. The link provided in the citation will take you directly to the reading or to a page where you can download it.

1. Schumacher, J. M. *Introduction to Financial Derivatives*. Open Press TiU.

<https://research.tilburguniversity.edu/en/publications/introduction-to-financial-derivatives>

- **Required pages:** Chapter 3, Section 3.2
- **Estimated reading time:** 15 minutes

NOTE: Content presented in the Required Readings and Lesson Notes will be tested in all assignments, including Quizzes.

LESSON 3

GENERATING CORRELATED STOCK RETURNS

Derivative Pricing - Module 5



Outline

- ▶ Generating correlated stock returns
- ▶ Cholesky decomposition

Generating Correlated Stock Returns

We already know how to simulate stock price paths randomly using the GBM equation:

$$dS = S (\mu dt + \sigma dW_t)$$

with the volatility component assumed in the form $dW_t = \epsilon \sqrt{dt}$, with $\epsilon \sim \mathcal{N}(0, 1)$ and independent process

⇒ What can we do if we want to simulate paths for various stocks correlated with each other?

- ▶ The underlying process (GBM) would be the same for all (with different drift and volatility parameters, probably).
- ▶ Our sampling of different ϵ should no longer be independent. The required correlated ϵ would be:

$$\begin{aligned}\epsilon_1 &= x_1 \\ \epsilon_2 &= \rho x_1 + x_2 \sqrt{1 - \rho^2}\end{aligned}$$

with x_i sampled from univariate standard normal and ρ being correlation coefficient.

Cholesky Decomposition

Cholesky decomposition (or factorization) will simplify a lot of the previous task.

⇒ **Cholesky factorization** is a decomposition of a positive-definite, symmetric matrix into a product of a triangular matrix and its conjugate transpose:

$$LL^T = C$$

This will allow us to create correlated multivariate normal random variables. These will be the steps:

1. Obtain correlation matrix for the desired stocks (historical?)
2. Obtain the Cholesky factor (L) (lower triangular matrix)
3. Generate vector of univariate standard normal values ($z \sim \mathcal{N}(0, 1)$)
4. Multiply the Cholesky factor matrix (L) by standard normal vector (z) to obtain our ϵ :

$$\epsilon = Lz$$

→ Please check the additional references and materials if you want to dig deeper into these concepts.

Summary of Lesson 3

In Lesson 3, we have looked at:

- ▶ How to simulate stock returns for a number of correlated assets
- ▶ Using Cholesky decomposition to generate correlated random numbers for the GBM process

⇒ **TO DO NEXT:** This lesson does not have an associated Jupyter Notebook. Instead, go to the next lesson to see a practical implementation of how to simulate correlated stock returns.

⇒ In the next lesson, we will develop a practical example in Python.

LESSON 4

AN EXAMPLE OF CORRELATED STOCK RETURNS

Derivative Pricing - Module 5



Outline

- ▶ Generating Correlated Stock Returns in Python

An Example of Generating Correlated Stock Returns

In this lesson, we are going to work on a fully developed example in Python of how to generate correlated stock returns under the GBM process.

The simulation of correlated price paths for a variety of stocks is very useful for a number of things, for example:

- ▶ Basket options: The underlying asset in this case is a "basket" of assets.
→ Hence, the payoff depends on the correlated evolution of n assets.
- ▶ Value-at-Risk (VaR) simulation: This allows us to simulate the behavior of a portfolio of stocks.
→ Calculate maximum portfolio loss with statistical confidence.

⇒ Go to the associated Jupyter Notebook for this lesson now to work on this.

Summary of Lesson 4

In Lesson 4, we have looked at:

- Using Cholesky decomposition to generate correlated random price paths with GBM process in Python.

⇒ **TO DO NEXT:** Go to the associated Jupyter Notebook for this lesson if you haven't done so yet.

⇒ In the next module, we will start taking a look at more complex models that try to bring things closer to real-life observations. For example, we will look at how we can price derivatives when we allow volatility to be time-varying, as observed in practice.

DERIVATIVE PRICING

MODULE 5 | LESSON 4

AN EXAMPLE OF CORRELATED STOCK RETURNS

Reading Time	50 minutes
Prior Knowledge	Stock returns, Cholesky decomposition, GBM
Keywords	Cholesky decomposition, Correlation

In this lesson, we will guide you through the process of generating correlated stock returns with the GBM process, using real data obtained from past returns' distributions and a correlation matrix.

This is going to be a very helpful tool when it comes to pricing, for example, a basket option.

- **What is a Basket Option?** → In short, an option whose payoff depends on the evolution of several underlying assets. More [here](#).

As always, let's start by importing the necessary libraries:

```
# Import libraries
%matplotlib inline
from math import exp, sqrt
import matplotlib.pyplot as plt
import numpy as np
```

1. Gathering Historical Data on Stock Returns' Distribution and Correlations

In order to simulate correlated stock price paths under the GBM process, we need some input parameters: volatility and correlation matrix. One possibility is to obtain these parameters from the distribution of past returns. Thus, our first step is to get this data. For illustrative purposes, we are going to use the parameters based on past historical returns of Apple, Amazon, Facebook/Meta, Google/Alphabet, and Microsoft.

```
import pandas as pd

file_paths = {
    'AAPL': 'AAPL_wqu_data.csv',
    'AMZN': 'AMZN_wqu_data.csv',
    'FB': 'FB_wqu_data.csv',
    'GOOG': 'GOOG_wqu_data.csv',
    'MSFT': 'MSFT_wqu_data.csv',
}

# Load adjusted close prices
df_prices = {}

for ticker, path in file_paths.items():
    df = pd.read_csv(path, parse_dates=['timestamp'])
    df.set_index('timestamp', inplace=True)
    df_prices[ticker] = df['close']

# Combine into one DataFrame
df_tickers = pd.concat(df_prices.values(), axis=1)
df_tickers.columns = df_prices.keys()

# Drop rows with any NaNs to ensure all tickers have data
# This forces the DataFrame to start from the first common available date
df_tickers = df_tickers.dropna()
df_tickers.head()
```

Next, we calculate the daily returns, volatilities, and correlation matrix of returns:

```
df_tickers_ret = df_tickers / df_tickers.shift(1) - 1
volatility = (
    df_tickers_ret.std()
) # This is daily volatility, we will later on annualize it!
df_tickers_corr = df_tickers_ret.corr()
print(df_tickers_corr)
```

2. Cholesky Decomposition

In the next step, we use Cholesky factorization in order to simulate our multivariate correlated normal variables (ϵ from slides).

For that, we make use of the numpy method `np.linalg.cholesky()`, which facilitates the task.

```
# Perform Cholesky decomposition on coefficient matrix
R = np.linalg.cholesky(df_tickers_corr)

# Compute transpose conjugate (only for validation)
RT = R.T.conj()

# Compare the R matrix and check the reconstruction of the original correlation matrix:
# 1. R matrix from Cholesky decomposition
print("This is the R matrix: ")
print(R)
# 2. Reconstruct coefficient matrix from factorization (only for validation)
print("This is the reconstructed original correlation matrix ")
print(np.dot(R, RT))
```

3. Simulate

Now, it is the time to make use of Cholesky decomposition in the previous point, and stock return volatility data to actually simulate the price paths for different assets taking into account the dynamics of the mentioned firms.

Remember, we do not really care about the μ parameter since we are in a risk-neutral world.

```
T = 252
Num = len(volatility) # Number of simulated days
stock_price_array = np.full(
    (Num, T), 100.0
) # Stock price, first value is simulation input
volatility_array = volatility * sqrt(252) # Volatility (annual, 0.01=1%)
r = 0.001 # Risk-free rate (annual, 0.01=1%)
dt = 1.0 / T # Time increment (annualized)
```

For comparability purposes, let's assume that $S_0 = 100$ for all of our different stocks simulated (obviously, this will change in real life for a basket option):

```
stock_price_array
```

With all the previous inputs, we can now simulate price paths using the “classic” price process:

$$S_t = S_{t-1} e^{\left(\left(r - \frac{\sigma^2}{2}\right)dt + \sigma \epsilon \sqrt{dt}\right)} \quad (1)$$

```
for t in range(1, T):
    # Generate array of random standard normal draws
    random_array = np.random.standard_normal(Num)

    # Multiply R (from factorization) with random_array to obtain correlated epsilons
    epsilon_array = np.inner(random_array, R)

    # Sample price path per stock
    for n in range(Num):
        dt = 1 / T
        S = stock_price_array[n, t - 1]
        v = volatility_array[n]
        epsilon = epsilon_array[n]

        # Generate new stock price
        stock_price_array[n, t] = S * exp(
            (r - 0.5 * v**2) * dt + v * sqrt(dt) * epsilon
        )
```

Finally, we can plot the paths:All rights reserved WQU WorldQuant University QQQQ

```
# Plot simulated price paths
fig = plt.figure()
ax = fig.add_subplot(1, 1, 1)
array_day_plot = [t for t in range(T)]
tickers = df_tickers.columns

for n in range(Num):
    ax.plot(array_day_plot, stock_price_array[n], label="{}".format(tickers[n]))

plt.grid()
plt.xlabel("Day")
plt.ylabel("Asset price")
plt.legend(loc="best")
```

```
plt.show()
```

4. Conclusion

Well done! Now you have learned how to use Cholesky's decomposition in order to be able to simulate price paths for a number of assets. In the next module, we will move forward in our understanding of derivative pricing by incorporating more complex pricing models where, for example, we start considering the case of a non-constant volatility parameter.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.